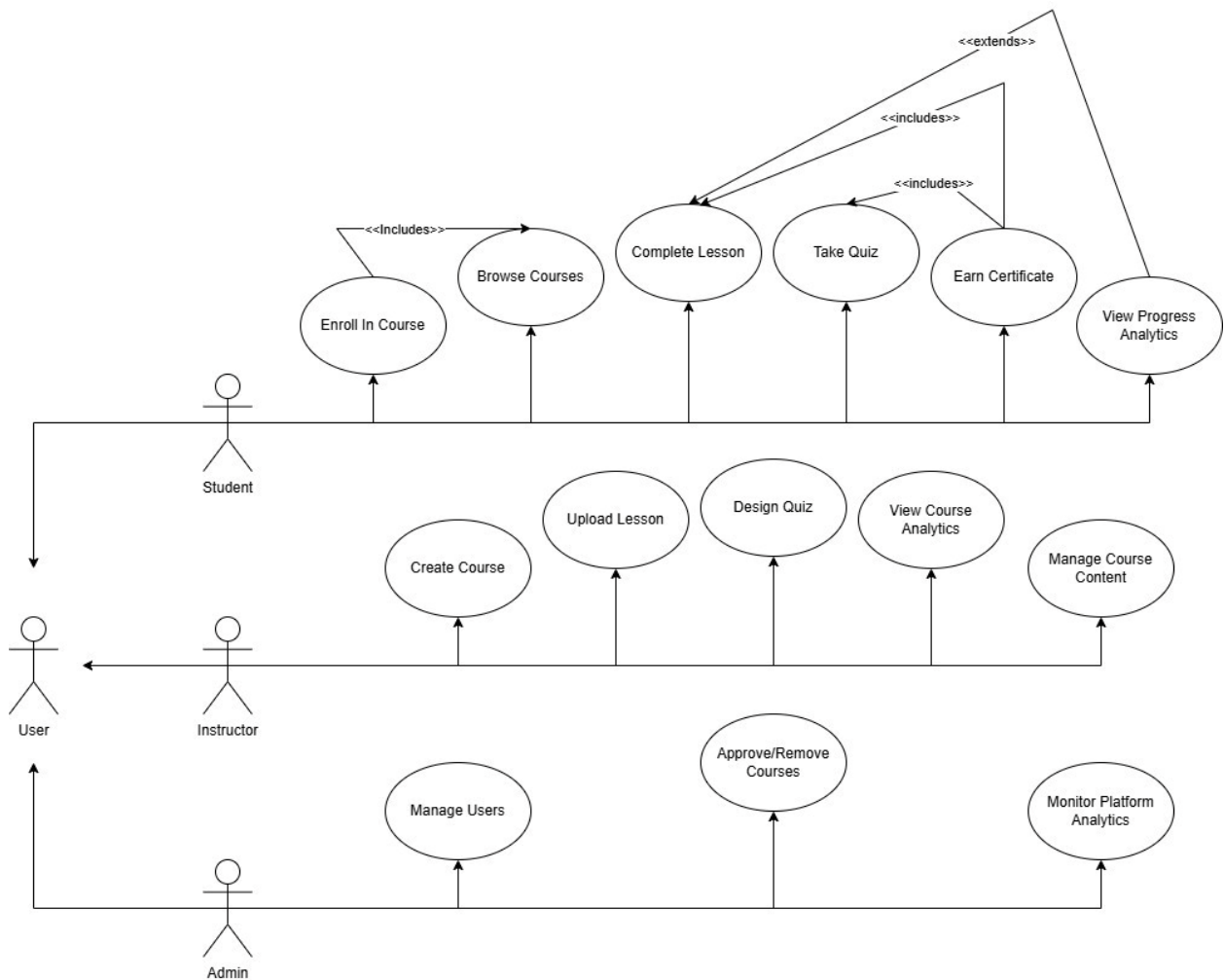
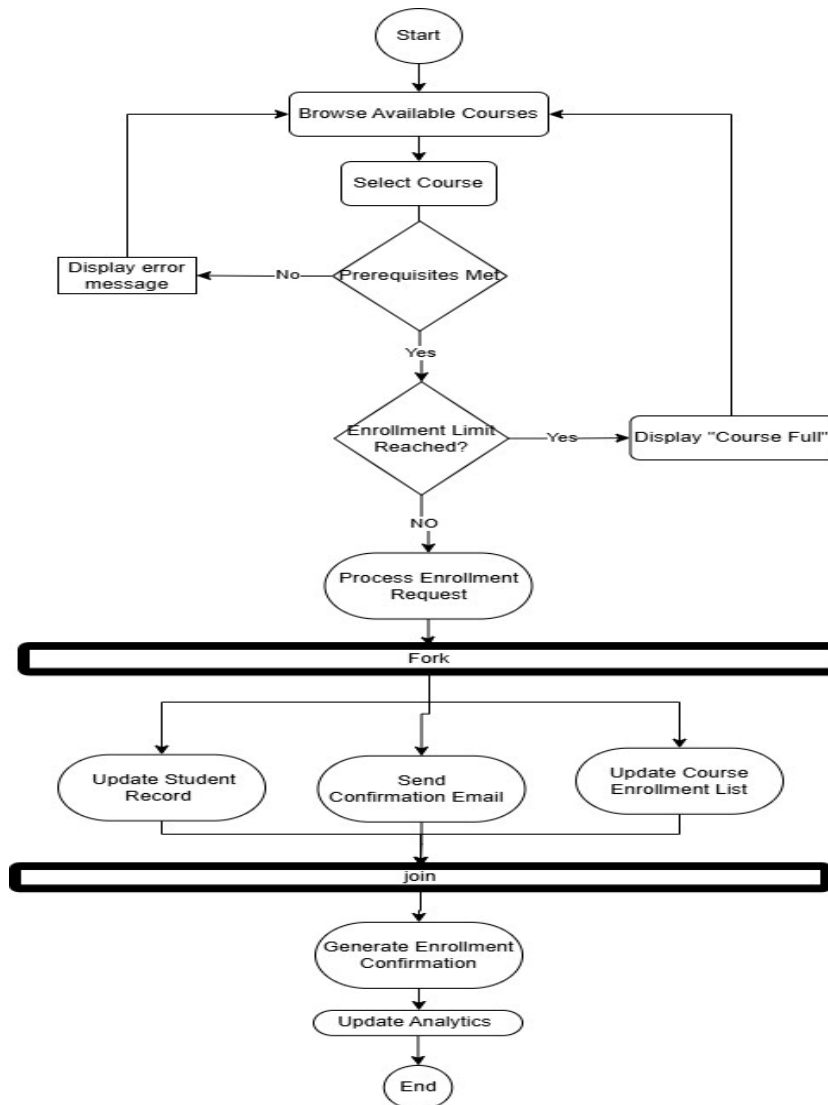


1. Use Case Diagram



This Use Case Diagram defines the functional scope of the SkillForge platform by mapping three actor types (Student, Instructor, Admin) to their respective use cases, demonstrating what each user can accomplish within the system. The diagram employs UML relationships including generalization to show that all actors are specialized types of User, <<include>> dependencies to represent mandatory sub-processes, and <<extend>> relationships to indicate optional functionality extensions. Together, these elements provide a complete specification of system features and actor interactions that will guide the implementation of role based access control and business logic

2. Activity Diagram:

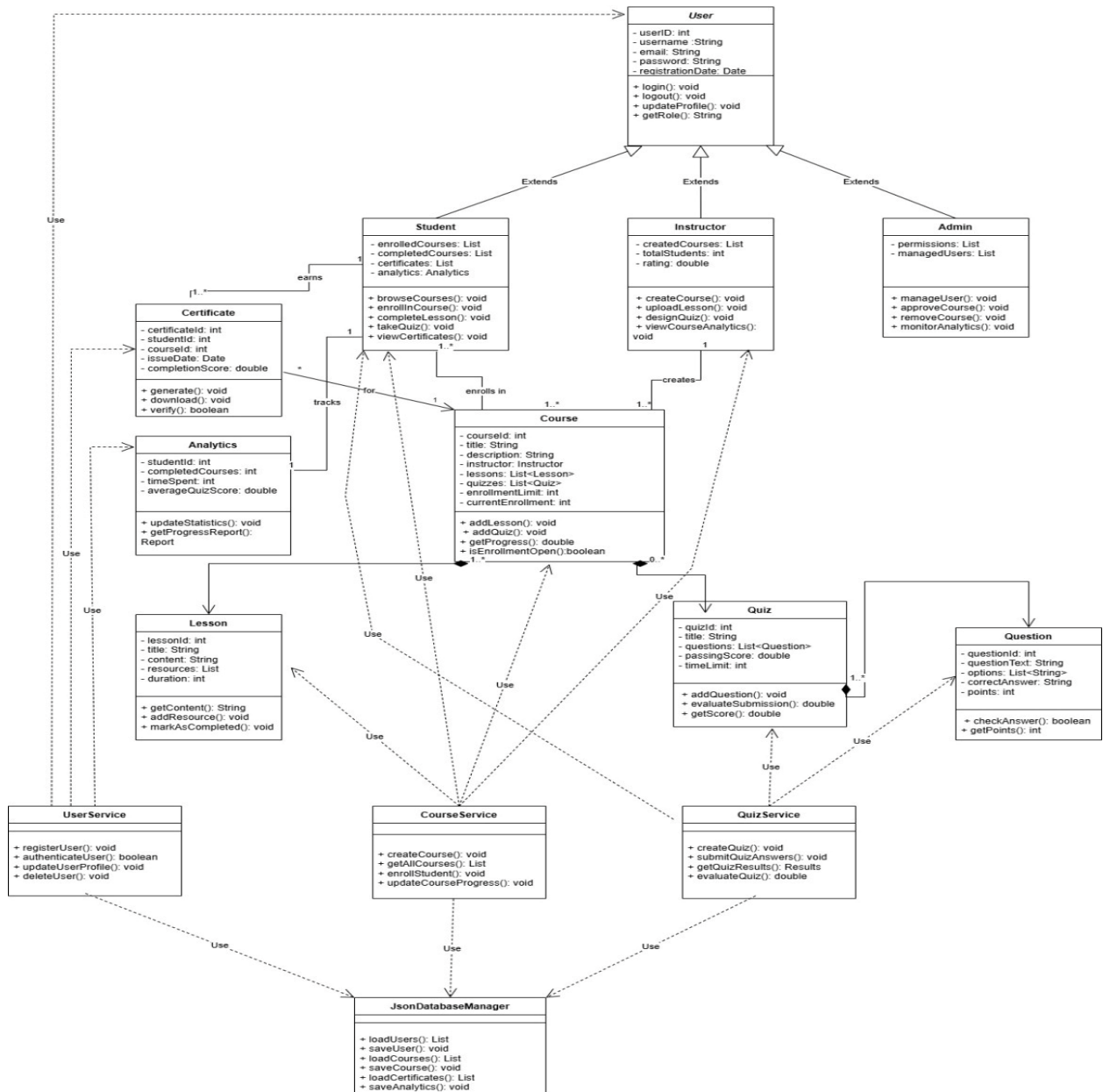


This Activity Diagram represents the business logic for student course enrollment, showing the control flow from initial course selection through validation , parallel data processing, and final confirmation

Decision nodes ensure that prerequisite requirements are met and enrollment limits are not exceeded , with appropriate error handling that redirects users back to course browsing

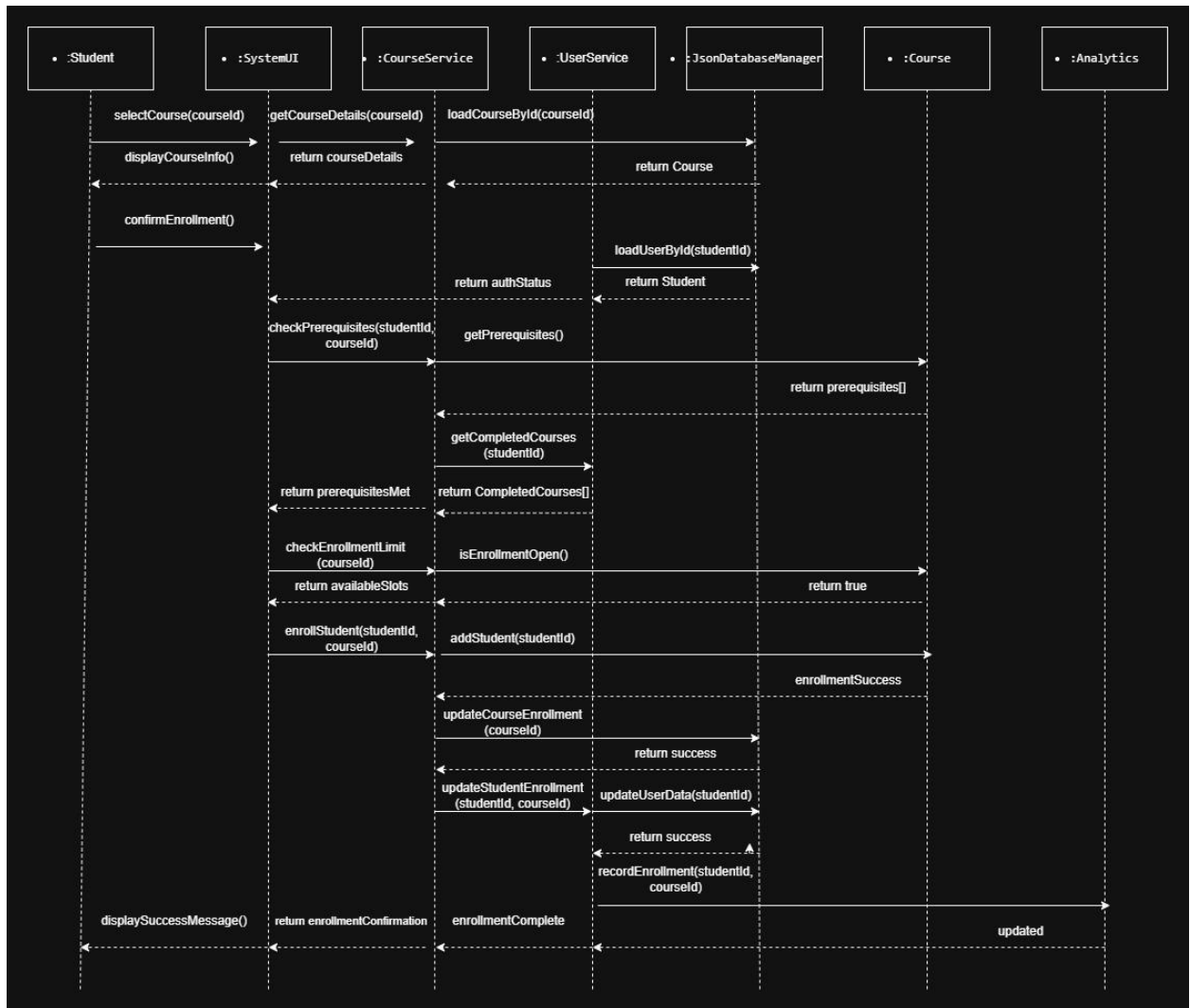
The fork/join synchronization pattern enables concurrent updates to student records , course enrollment data , and email notifications , ensuring data consistency before completing the enrollment process with analytics tracking

3. Class Diagram



The Class Diagram represents the static structure of the SkillForge platform, illustrating 14 key classes with their attributes, methods, and relationships. It demonstrates object-oriented design principles through inheritance (User hierarchy with Student, Instructor, and Admin subclasses), composition (Course containing Lessons and Quizzes), association (Students enrolling in Courses and earning Certificates), and dependency (Service classes using domain classes and JsonDatabaseManager for data persistence). This diagram serves as the comprehensive blueprint for implementing the system, ensuring proper separation of concerns between domain entities, business logic services, and data access layers.

4. Sequence Diagram



Drawed a sequence messages diagram showing the relationship between the 7 headers where sent functions are drawn in solid arrows but returned values in hashed arrows , where each hashed line defines one of the 7 headers with Neated and well organized way